

An Evidence-Bound Runtime Contract for LLM Narration of Machine-Learning Pipeline Evidence

Hakan Sabuniş · Yusuf Ünlü ·
Mehmet Kemal Özdemir

Received: date / Accepted: date

Abstract When a language model explains a deterministic analysis to an engineer, a column or number the analysis never produced can be acted on. We study this in **mlcompass**, an open-source assistant for machine-learning pipelines, whose language model narrates structured evidence, such as a leakage detector’s findings. For each field of the narrator’s answer the evidence fixes what may appear there; we bind those admissible sets at call time and check every answer in host code, sending back what fails with the problem named. The mechanism is ordinary validate-and-reask; our contribution is what it is up against. Across three models, sixteen leakage tasks on synthetic and public data, and a dataset-profiling task, the model that fails almost never invents a column. It misfiles: it places something the evidence really contains, above all the name of the metric under investigation, in a field that does not admit it. A one-sentence field description, the shipped prompt, a lower temperature or another provider each remove most of this, and it returns wherever the fix is missing. A list of allowed columns written in advance turns misfiling into invention once the data changes; one built from the evidence at each call does not. In a well-specified configuration the verifier rarely had anything to catch, and whether it then adds a measurable benefit is a question our registered confirmatory test left open. What it adds is a check that holds whatever the configuration, at negligible cost. Verdict and free-text prose lie outside it.

Keywords Large language models · faithfulness · runtime verification · structured output · constrained generation · data leakage

H. Sabuniş (corresponding author) · Y. Ünlü · M. K. Özdemir
School of Engineering and Natural Sciences, Istanbul Medipol University, Istanbul, Türkiye
E-mail: hakan.sabunis@std.medipol.edu.tr

1 Introduction

Machine-learning practice runs on single-purpose tools: profilers (YData 2026), experiment trackers such as MLflow and TensorBoard (Zaharia et al. 2018; Abadi et al. 2016), and automated model search (Feurer et al. 2015). None of them tells the engineer what the results *mean*. The now-standard answer is agentic: a large language model (LLM) calls the tools, reads their structured output and explains it (Yao et al. 2023; Schick et al. 2023; Anthropic 2024). LLMs produce content their input does not support (Ji et al. 2023; Huang et al. 2023), and assistants built on them still fail real software tasks (Chen et al. 2021; Jimenez et al. 2024).

A concrete case shows what is at stake. An engineer’s regression model scores $R^2 = 1.0$ on held-out data. **mlcompass**, the open-source machine-learning (ML) pipeline assistant this paper studies, treats a score that high as a sign of data leakage: it fires, for example, when $R^2 > 0.999$, an area under the receiver operating characteristic curve (AUC) > 0.995 or accuracy > 0.99 on at least 50 rows, thresholds of our own motivated by the leakage problem Kaufman et al. (2012) describe. A deterministic detector then computes how strongly each feature correlates with the target and ranks the columns that could be carrying the target into the features. Its output is the *evidence*, a dictionary we write E . An LLM explains E to the engineer by filling in a structured answer: a verdict chosen from four labels (leakage likely, leakage uncertain, score legitimate, cannot determine), the list of columns it cites, and claims of the form (column, statistic, value). On the frame this paper studies (Section 5), a sound answer cites the leak column `log_target_v2` with its measured correlation of 0.999.

The narrated object is thus *structured evidence*, and a structured claim about it can be wrong in two ways. It can name an entity or quote a quantity that appears nowhere in the evidence (an *extrinsic* error). Or it can put something the evidence really contains where it is not admissible, such as the metric’s name `r2` in the list of cited columns, or one column’s measurement on another (an *intrinsic* error, which we call *misfiling*). On the model that fails in this study, 1,807 of the 1,829 flagged responses under the bare instruction, on sixteen task instances, are misfilings, and they come from a field whose admissible set nobody stated. One sentence of field description or the prompt **mlcompass** ships removes it, temperature 0 nearly does, and two other providers never produce it; where no fix is present, it returns.

Each fix works where it was measured, but a tool author cannot measure every combination of prompt, schema, user data, provider and date the tool will meet. What the narrator may cite, however, is *finite and machine-enumerable at the moment of the call*.

Definition 1 (Evidence-closed narration) A narration task is *evidence-closed* if the set of citable entities and verifiable quantities is finite and machine-enumerable at the moment of the narration call.

For such a task, checking the structured claims against the evidence takes a membership test per field and a numeric comparison per claim, so their soundness can be enforced instead of measured. We keep to this narrower scope throughout. A free-text sentence can name only real columns and quote only real values and still assert a causal story the evidence does not support, and the evidence itself can be wrong.

Constrained decoding (Willard and Louf 2023; Beurer-Kellner et al. 2023; Geng et al. 2023; Lundberg and Ribeiro 2023; Scholak et al. 2021) needs the token distribution, which commercial APIs withhold. Each payload returned under a provider-side schema (OpenAI 2024) can be checked against it, but how the provider produced a conforming payload, and whether it will for the next request, cannot be seen from outside. Our mechanism is an **evidence-bound runtime contract**. The narrator answers by calling a tool whose input is described by a JSON Schema, and a schema’s `enum` lists the only values a field may take. *Tier A* builds these lists from the evidence at call time, so the column field admits exactly the columns in E ; an enum is a request to the model, not a check. *Tier B* re-checks the returned payload in plain code for entity soundness, value soundness of {column, statistic, value} claims, and coverage of the top-ranked candidate, and answers a violation with a named corrective retry, residual stripping and omission flagging. Toolkits such as Instructor (Liu 2023) and Guardrails AI (Guardrails AI 2026) already support this validate-and-reask pattern, and we do not claim the mechanism as new. It is the default narration path of mlcompass (MIT licence), where it guards the leakage investigation described above; static leakage detectors for notebook code address the same problem before training (Yang et al. 2022; Truong et al. 2025). We describe it as shipped rather than in production, because we have no deployment telemetry.

We ask five research questions (RQs). **RQ1**: how does an unconstrained narrator of evidence-closed material fail, and what produces the failure? **RQ2**: does the contract prevent it on the checked channels, and at what cost? **RQ3**: how do the alternatives a practitioner would reach for compare, namely a stock guardrail toolkit, a provider schema, an enum written in advance and a rule-bearing prompt? **RQ4**: how far do instruction wording and specification move compliance? **RQ5**: do the contract and the failure carry over to a second kind of evidence?

The paper makes four contributions. (i) It characterises how an unconstrained narrator of structured evidence fails. On the one model of three that fails, across sixteen leakage instances, the failure is misfiling caused by an under-specified field, not invention; cheap fixes remove it where they are applied, and it returns where they are absent. (ii) It formulates field-indexed admissibility for evidence-closed narration (Definitions 1 and 2), which separates misfiling from invention, serves as the scorer, and lets a generic membership check enforce soundness. (iii) It compares the alternatives a practitioner would reach for inside one guardrail toolkit and one provider, and measures what a verifier adds to a well-specified configuration: nothing observable on leakage and a small exploratory catch on profiling, for microseconds per response. An

author-time enum applied to data it predates does worse than no enum, and the toolkit’s own choice semantics bound to the evidence match our validator on the field they check and miss what they do not check. (iv) It provides a replication package (Sabuniş et al. 2026) with every model reply, one record per live response, the frozen analysis plan with its amendments and deviations, the registered falsifier, and fifteen instrument faults with what each moved.

2 Background and Related Work

2.1 Closed-world faithfulness, and where it has been measured

Definition 1 is a closed-world assumption (Reiter 1978) over E : whatever the evidence does not carry is not admissible under the contract. The setting is old. Data-to-text and meaning-representation-to-text generation narrate a finite record and measure faithfulness against it offline, with slot error rate over the input’s slots (Dušek et al. 2020), relation-generation precision against the source table (Wiseman et al. 2017), and table-grounded scoring (Dhingra et al. 2019). In the taxonomy of Ji et al. (2023) and of Maynez et al. (2020), a name absent from E is an *extrinsic* error and a real evidence item asserted in the wrong place is an *intrinsic* one. We use *invented* and *misfiled* for these two classes on a structured channel and claim neither category as new. Xu (2025) argues that hallucination is unavoidable under an open world and can be mitigated under a closed one, Luo et al. (2026) repair a model’s own explanations by removal, and AVA (Karnatak et al. 2026) verifies citations against a curated report corpus and abstains when it cannot support an answer. An evidence-closed tool narrator changes only where the check runs: at call time, on the payload the user will see, instead of offline on a corpus.

Tool-use frameworks (Yao et al. 2023; Schick et al. 2023; Anthropic 2024) govern how an agent acts and say nothing about what it may afterwards claim about a tool’s output. Provenance work reconstructs evidence support from traces after the fact (Wang et al. 2026), and Ng et al. (2026) argue that agent safety belongs in a runtime contract. The enforcement mechanism itself is classical. Tier B is a postcondition in the sense of Design by Contract (Meyer 1992), and deleting what violates it is *suppression*, the weakest edit an edit automaton can make (Ligatti et al. 2005). Schneider (2000) and Falcone et al. (2012) characterise which properties a monitor can enforce at all.

2.2 Where the enforcement lives, and when it is bound

Decode-time methods can bind constraints to the input as well. Grammar-constrained decoding admits input-dependent grammars (Geng et al. 2023). PICARD, a constrained decoder for text-to-SQL, and Synchronesh, its counterpart for code generation, check partial programs against a database schema or a program’s scope (Scholak et al. 2021; Poesia et al. 2022); monitor-guided

Table 1 Ways to constrain what an LLM may output. *Locus*: where the constraint acts: while the model picks tokens (decoding), in the provider’s handling of a schema, or in code that checks the returned answer. *Bound*: when the allowed set is fixed, when the tool is written (author) or for each request from the data (call). *Logits*: whether the method needs the model’s token probabilities, which commercial APIs do not expose. *Guarantees*: what holds for every output. This paper’s row shares its locus and binding time with validate-and-reask tools; the differences are the field-indexed domain derived from a deterministic producer, and what we measure.

Locus	Bound	Logits	Guarantees	Example
Decode, grammar	author/call	yes	grammar	Willard and Louf 2023; Geng et al. 2023; Scholak et al. 2021; Poesia
Decode, span	call	yes	verbatim spans	Signé et al. 2026
Decode, closed loop	call	yes	none (empirical)	Cruz 2026
Provider schema	author/call	no	structure, uncertified	OpenAI 2024
Code, policy	author	no	policy contract	Ahn and Kim 2026; Zhang et al. 2025b; Winston et al. 2026
Code, context + reask	call	no	what the validator checks	Liu 2023; Guardrails AI 2026
Code, ledger	accumulating	no	per-claim provenance	Du et al. 2026
Code, field-indexed	call	no	(C1)–(C2), (C3) flagged	this paper

decoding consults static analysis of the repository (Agrawal et al. 2023); and CHyD (Signé et al. 2026), a decoder for retrieval-augmented generation, restricts output to spans of the retrieved context. All of them need the token distribution, which a commercial endpoint withholds. ATLAS-RTC (Cruz 2026), a token-level runtime controller for agent output, intervenes during decoding with masking and rollback, and reports empirical gains without a guarantee. Le (2026) shows that schema-key wording moves accuracy in both directions, and JSONSchemaBench (Geng et al. 2025), which measures constrained-decoding frameworks on 10,000 real-world schemas, finds that they differ in which schema features they cover. What a schema enforces therefore depends on the engine applying it, which is a reason to distrust a schema whose effect has not been checked. Nor does a valid schema make an answer right: on OrderBench (Li 2026), the strongest of four ordering agents produces schema-valid output every time and still gets about one order in five wrong.

Other tools already sit in this paper’s row of Table 1. Instructor (Liu 2023), a Python library for structured LLM output, validates a response against context supplied at call time and re-asks on failure. Guardrails AI (Guardrails AI 2026), an open-source validation framework, and NVIDIA’s NeMo Guardrails (Rebedea et al. 2023) ship validate-and-reask loops whose validators can read call-time metadata. Our mechanism is one instance of that practice. We add three things: the field-indexed admissible domain, derived from a deterministic producer’s output, which separates misfiling from invention and which a generic membership check can then enforce; measurements of which failure actually occurs, what enforcement costs and which alternatives reach the same result; and a shipped artifact whose narration path is the one we measured.

Three pieces of work are closest. Ahn and Kim (2026) vary the enforcement layer with the model fixed and find what we find: prompts let violations through, a bolt-on guardrail over-refuses (88/120 utility against 120/120), and code-owned enforcement keeps both, over 270 runs on three hosted models.

Their contract is a policy fixed when the harness is written, whereas ours has no admissible domain until the evidence exists. The same holds for RvLLM (Zhang et al. 2025b), which checks outputs at runtime against constraints that domain experts write in a specification language, and for Winston et al. (2026), who translate natural-language policies into satisfiability-modulo-theories constraints and check each tool call before it runs. LedgerMind (Du et al. 2026) lets claims cite only active ledger entries, checks entity and numeric grounding, and restricts repair to transitions that add no unprovenanced content. That covers three of our channels and our repair asymmetry (Observation 1), which we derived independently; its ledger grows during a trajectory, while ours is complete before the call. EviBound (Chen 2025) gates an ML research agent by querying MLflow for a run, its artifacts and a terminal status, with bounded retries, eleven months before our measurements. It checks that a claimed *action* occurred, for instance that a training run the agent reports really finished. We constrain what may be *said* about a measurement already taken: the narrator may report `log_target_v2`'s correlation as 0.999, but may not list `r2` as a column or quote a correlation E does not contain.

3 Evidence-Closed Narration

3.1 The evidence, and why admissibility is per field

In the leakage investigation, the evidence dictionary E holds the suspicious metric and its value; per-feature correlations against the target y , taking whichever of ρ_P and ρ_S has the larger magnitude and keeping its sign; a ranked candidate-leak set $C = \{c : \max(|\rho_P(c, y)|, |\rho_S(c, y)|) \geq 0.99\}$; and the perfect-match rate $P(\hat{y} = y)$. We report Spearman because monotone target transforms pull ρ_P down while ρ_S stays at 1. Our log-target leak sits at $\rho_P = 0.93$ against $\rho_S = 0.999$ and would slip under a Pearson-only filter.

Let N_E be every name E carries, its keys and its string values. The obvious formalisation admits any member of N_E in any reference field. Under it, a column field reading `r2` counts as sound, because `r2` is in E as the name of the *metric*. We therefore index admissibility by field.

3.2 The contract, stated as an invariant

Let F be the answer's reference-bearing fields: in the leakage contract, the list of cited columns and the column and statistic of each claim (`columns_referenced`, `claims[].column` and `claims[].statistic` in the schema). Their entries, the *leaves*, are names compared as strings. E determines an admissible set $A_{E,f}$ per field and a value table V_E with $\text{dom}(V_E) \subseteq A_{E,\text{col}} \times A_{E,\text{stat}}$. The sets of different fields may share a name, as when a frame has a column called `mean`; each is checked in its own field. A leaf outside the $A_{E,f}$ of its field is *misfiled* when it lies in N_E . Otherwise it is outside the evidence, and for measurement we split that case by the data: a real column of the frame that E

does not list is *unlisted*, and a name that exists in neither E nor the frame is *invented*. The verifier sees only E and treats all three alike. A response *commits* when its verdict is admissible and is not the abstention “cannot determine”; the entities it *references* are its leaves in the column fields.

Definition 2 (Evidence-bound runtime contract) A structured response r satisfies $\mathcal{C}_E$, with tolerance τ and anchor a^* , if **(C1, entity soundness)** every leaf of r in a field $f \in F$ lies in $A_{E,f}$; **(C2, value soundness)** every claim (e, s, v) in r has $(e, s) \in \text{dom}(V_E)$, v a finite number, and $|v - V_E(e, s)| \leq \tau$; and **(C3, anchor coverage)** if r commits, the anchor a^* , the top-ranked candidate, is among the entities r references.

(C1) and (C2) together state *referential and value soundness with respect to E* on the structured channels. This is narrower than faithfulness in general: the verdict, the free-text narration and the suggested checks lie outside it, and E itself is trusted. Tier B enforces (C1) and (C2). A response that violates either is sent back, at most twice, with the violation named, and whatever still violates it after that is deleted. A response with no admissible verdict is retried the same way and, if the problem persists, aborted instead of completed on the narrator’s behalf. Hence *no structured claim that Tier B returns violates (C1) or (C2)*; this follows from the deletion step (Observation 1). Whether our Python implementation realises the algorithm is a separate, empirical question. Thirty-six mutation tests and a property-based search over 10^6 generated (evidence, response) pairs, registered in the analysis plan before the study, support it without proving it, and the search found two defects in an earlier version (Section 10). Because the guarantee does not come from sampling, we report an enforced arm’s clean result as what the verifier did and attach no interval to it. (C3) cannot be enforced in the same way, so it is reported and not guaranteed: a committed response that still omits the anchor after the retries reaches the user with the omission flagged. When E has no candidate there is no anchor, and (C3) holds vacuously.

Observation 1 (Repair asymmetry) *Soundness is safe-repairable by deletion: the projection $\Pi_E(r)$ removing every violating element is sound, idempotent, and the identity on sound responses. Anchor coverage is not: any repair must add a reference the model did not produce.*

Proof sketch Deletion preserves membership and tolerance and can remove every violating element. For anchor coverage, deleting the verdict would discharge the obligation, but the schema requires a verdict, and substituting “cannot determine” would attribute to the narrator a stance it did not take. Both premises are stipulations of this contract, and the artifact honours them by aborting instead of substituting. Given them, every remaining repair inserts an anchor reference absent from r .

Tier B therefore strips unsound content and only *flags* a persistent omission, which in edit-automaton terms is suppression without insertion (Ligatti et al.

2005). In the security setting insertion is simply a stronger primitive; over a narrator, an inserted reference would fabricate attribution. With each $A_{E,f}$ and V_E hashed once, a response costs $O(|r|)$. The step that needed care was keying V_E on the pair, which our own scorer once failed to do (Section 10).

4 The Artifact

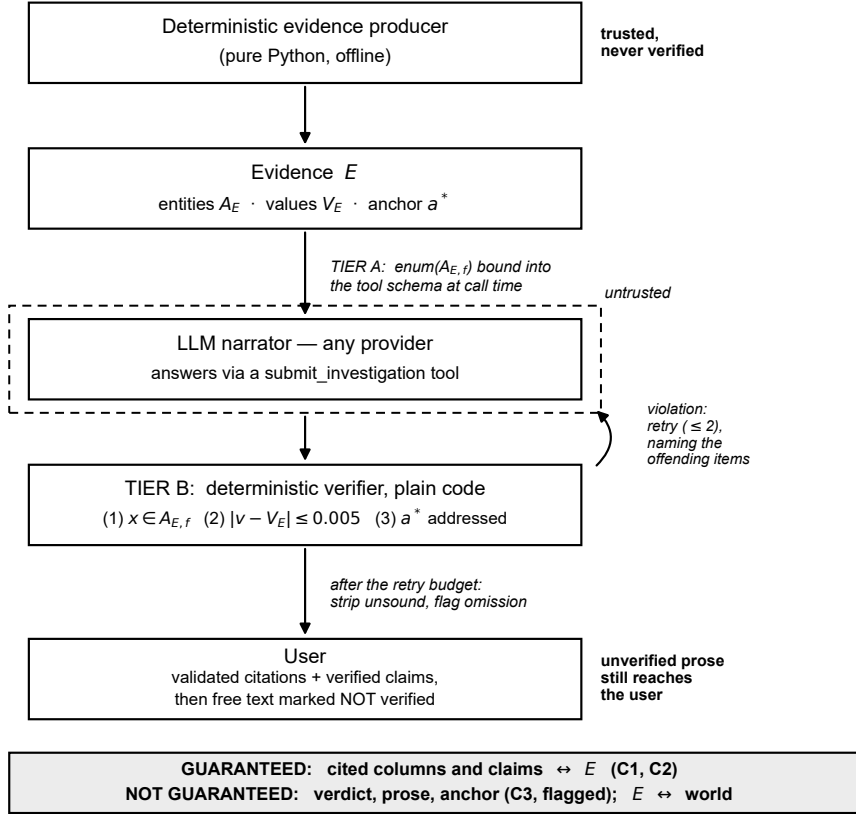


Fig. 1 The evidence-bound contract. Tier A binds the tool schema’s enums to E at call time. Tier B re-checks the returned answer in host code, retries with the offending items named, and after the budget strips unsound content and flags a persistent omission. The dashed boundary encloses only the narrator. The producer is trusted and never verified, the free-text field reaches the user marked unverified, and the guarantee is consistency with E , which says nothing about whether E matches the world

Algorithm 1 Evidence-bound narration (budget $k = 2$)

```

1:  $B \leftarrow \text{bind}(E)$   $\triangleright A_{E,f}, V_E, \tau, a^*$ 
2:  $S \leftarrow$  schema with enums from  $B$   $\triangleright$  Tier A
3:  $c \leftarrow$  empty correction
4: for  $i = 0, \dots, k$  do
5:    $r \leftarrow \text{model}(\text{prompt} + c, S)$   $\triangleright$  provider errors propagate
6:   if  $r$  has no admissible verdict then
7:      $c \leftarrow$  fixed correction; continue
8:   end if
9:    $\text{viol} \leftarrow \text{verify}(r, B)$   $\triangleright (C1), (C2), (C3)$ 
10:  if  $\text{viol} = \emptyset$  then break
11:  end if
12:   $c \leftarrow$  correction naming  $\text{viol}$ 
13: end for
14: if  $r$  has no admissible verdict then return abort  $\triangleright$  evidence panel only
15: end if
16:  $r \leftarrow \text{strip}(r, B)$   $\triangleright$  delete leaves violating (C1), (C2)
17:  $\text{flag} \leftarrow r \text{ commits} \wedge a^* \neq \perp \wedge a^* \notin \text{refs}(r)$ 
18: return  $(r, \text{flag})$   $\triangleright (C3)$  flagged, never repaired

```

4.1 Architecture and the trust boundary

Figure 1 splits the system at a trust boundary. One side holds everything deterministic: the pure-Python evidence producer, E , the Tier B verifier and the user-facing output. The other side holds the LLM narrator, which we treat as untrusted whoever serves it. The system has three *layers*: evidence production, a narrator told the rules in words (Section 7 measures what that is worth), and the evidence-bound schema, through whose submission tool the answers arrive. The contract spans the last two layers as two enforcement *tiers*. Tier A restricts the cited-column list and each claim’s column to enums built from E at call time. It remains a request. On **deepseek-chat** every Tier A arm in the main battery ran without the provider’s strict decoding flag, and we report the one arm run with the flag separately. Tier B re-checks the returned tool input in plain code for (C1), for (C2) within $\tau = 0.005$, and for (C3).

Algorithm 1 gives the control flow. Anchor coverage is checked after stripping, because deleting an unsound claim can orphan the anchor. A response with no admissible verdict (no tool call, unparsable arguments, or a verdict outside the enum) is aborted after the budget, and no verdict, claim or prose is shown. The deterministic evidence panel is printed in every case and lists the candidate leak columns with the anchor first, so neither an abort nor a flagged omission hides it. The abort path is recent; the earlier code passed such a response as an empty answer. In the records they occur only on arms without a verifier (5 on **deepseek-chat**, 8 on **qwen2.5:7b**), so no reported number depends on the change.

4.2 One contract, two instances

For most of this study the code handled a single instance: one column set, each claim’s statistic fixed to correlation, and each claim checked by looking its column up in the correlation map. The contract is now a layer. A specification says how to read one evidence shape, binding yields the field-indexed domains, the value table and the anchor, and the verifier, stripper, retry text and schema builder name no task. Leakage is one specification of about forty lines. A dataset profiler is a second: every column of the frame is admissible, each column has up to thirteen statistics, and the statistic set is computed per call because a categorical column carries a cardinality and no mean. Each specification also names its anchor: the top-ranked candidate for leakage, and the column with the most missing data for the profile. The profile narrator runs in the measurement harness, and the leakage narrator is the one the product ships.

Extracting the layer exposed something the single task could not: the value table must be keyed on (entity, statistic). With one statistic per column the two keyings coincide. In a profile, the column **age** has mean 38.5 and missing fraction 0.012, and a table keyed on the column alone checks a claim about one against the other. That is one of the scorer defects in Section 10, reappearing in the production path. The evidence’s surface shape also has to match the flat claim slot: narrators handed a nested quantity invented a flattening, five different ones in all.

4.3 What the invariant does not cover, and a threat model

Four things fall outside the invariant. The first is the verdict and the prose. The verdict is checked for admissibility and not for agreement with E , so a narrator may cite the planted leak and still answer “score legitimate”. None did: of the 8,363 responses whose verdict is one of the four labels on instances with a leak, planted or real, 8,263 say “leakage likely”, 54 say “leakage uncertain” and 46 abstain. On the negative control none of 799 says “leakage likely”: 381 abstain, as the shipped prompt asks when there is no candidate, 238 say “uncertain” and 180 “score legitimate”. That is an observation and not a property of the contract. The free-text narration and the suggested checks are printed beneath the validated content and marked unverified; Section 12 measures what they carry.

The second is the correctness of E . If the leakage detector mis-ranks a candidate, the contract faithfully narrates a wrong finding. The producer is trusted and not verified, and (C3) names one anchor rather than every finding. A response that addresses the second of two nearly tied candidates would be flagged; no instance we ran has such a tie.

The third is the other narration surfaces. The product prints two other narrations of the same evidence that no contract governs: an optional LLM interpretation panel printed by the evaluation command, and a tool-server

command whose calling model narrates in the user’s editor (Anthropic 2024). The interpretation panel is marked unverified. The tool-server narration carries no mlcompass marker at all, because the client model writes it. mlcompass guarantees the contract panel and nothing else.

The fourth is adversarial evidence. Column names come from the user’s data and reach the schema’s enums, the prompt, the corrective retry and the rendered panel, and membership in E does not make a string safe. A crafted header is a prompt-injection vector (Greshake et al. 2023); schema keys and enums are part of the model’s instructions (Le 2026), and through Tier A’s enum values a header reaches the channel that the EnumAttack of Zhang et al. (2025a) uses to carry a harmful request past a model’s safety training. The contract guarantees only that what is shown is in E , and rendered strings are escaped. Capping name length, restricting names to an identifier-like character set and quoting names as data would each reject or alter some legitimate headers; we have neither implemented nor measured them.

5 Study Design

Following the ACM SIGSOFT Empirical Standards (Ralph et al. 2020), the community checklists for what an empirical software-engineering study should report, the study is a controlled experiment over fixed evidence dictionaries. The manipulated variable is the enforcement mechanism (RQ1–RQ3, RQ5), the instruction wording (RQ4), or the naming and description of the evidence (the controls of RQ1 and RQ5). The analysis plan and its amendments were frozen before the runs they govern and are in the replication package, and Table 2 reports what the plan registered and what became of each item.

5.1 Subjects

The subject is the shipped leakage-investigation path. The *reference* task is a 1,000-row frame of fourteen columns (target, prediction and twelve candidate features). The target is $y \sim \mathcal{N}(50, 10^2)$, and the frame carries a monotone transformed leak, $\log_target_v2 = \log(y - \min y + 1) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 0.01^2)$, the kind of column left behind when a log-transformed copy of the target stays among the features, and a high-correlation distractor. Its evidence has ten columns and the anchor `log_target_v2`. A second, *crowded* instance has 1,200 rows, an anchor `sensor_ref` correlating at 0.995 with the target, and nine near-tied distractors `sensor_00...sensor_08` at 0.900–0.924. A column list written while authoring against the reference task covers none of its ten evidence columns. Two further instances are real data, each narrated after an ordinary least-squares fit on all numeric features. *Bodyfat*, 252 men (Penrose et al. 1985), has a documented leak: body fat is computed from body density by Siri’s equation, so the fit reaches $R^2 = 0.978$ and the detector names `Density` ($|\rho| = 0.993$) as the only candidate. *Sambanis* is the civil-war onset data of

Table 2 What the analysis plan predicted, and what happened. X-items are planned comparisons between two arms; X1, X3, X4 and X5 form the Holm-corrected family. H-items are hypotheses, numbered as in the plan (H1–H6 belonged to an earlier round). Differences are in percentage points (pp) of the rate of responses reaching the user with any violation, on the 2026-09-15 runs, with Newcombe 95% intervals and Holm-adjusted Fisher tests. The five rows after the first rule record where the study departed from the plan; the last four are amendment A3, registered for this revision.

Item	Prediction	Outcome
X1	Contract below the stock Guardrails loop	Stock loop higher by 35.0 pp [28.5, 41.8], $p_{\text{Holm}} < 10^{-23}$; registered on a real-data task, where only the floor, shipped and strong arms were later run; reported on the synthetic one
X2	Contract below the strict call-time enum on a channel other than entity	Void: no such channel activated on leakage
X3	Contract below a typed schema without column list	Schema higher by 4.0 pp [1.3, 7.7], $p_{\text{Holm}} = 0.015$; that arm ran without the strict flag
X3b	Contract against the author-time list (exploratory)	Equal on the reference task; 72.5 against 0% on the crowded instance
X4	Our checks inside the Guardrails loop tie the contract	0.0 pp [−1.9, 1.9], $p = 1$
X5	Contract against NeMo Guardrails	Not run: the adapter was not built
H7	Stock Guardrails loop within 2 pp of the floor	Not met: 35.0 against 42.0%, but not the 10 pp drop that rejects it
H8	Typed schema without column list within 2 pp of the floor	Neither held nor rejected: the registered rule rejects only at 0/200, so it cannot register the large drop observed (4.0 against 42.0%)
H9	Strict call-time enum ties the contract on entity	Held (2026-09-24); the run without the flag is A-TIER-A-ONLY
H10	Strict enum fails on an activated non-entity channel	Void with X2
H11	Our checks inside Guardrails match the contract on every channel	Held
H12	Wording effects differ between providers	Not run: the sweep ran on one provider
H13	Every contract arm reads 0 on every provider	Held
H14	The value channel activates under designed pressure	Not on leakage; the profile task activated it outside the registered design
H15	The omission channel activates under low anchor salience	Not run
H16	The post-strip anchor-coverage re-check fires on live data	Not exercised: nothing was stripped
Falsifier	Property-based falsifier with automated mutants	Built after the runs (10^6 cases); 36 hand-written mutation tests instead of generated mutants
Strict prompt	Replicate of the arms under the shipped prompt	Run for A-L2 and A-L3-SHIPPED only, plus the unregistered A-L2-RULES and A-L2-NOCLAIMS
Panel	deepseek-v4-flash, gpt-5.4-mini, Qwen2.5-3B (vLLM)	deepseek-chat, gpt-5.4-mini, qwen2.5:7b (Ollama), without an amendment
Pooling	No pooled rates for floor, baseline or sweep arms	None, except the share of invented names over the twelve A3 instances, which A3 registered pooled
Pilot	Report the June floor rates	15.0 and 11.5% (rolling alias, records not kept; fault 5)
A3 R2a	On 12 instances frozen in July, misfiling dominates and invented names stay under 5% of flagged responses	Held: 12 of 817 (1.5%), on 1,888 valid floor responses
A3 R1, R2b	The verifier lowers delivered violations in a well-specified configuration (profile, $N = 600$; 12 instances)	Not run: the provider balance ran out (A4); partial cells reported descriptively
A3 R3	Measure the shipped default narrator	Not run: no budget
A3 R4	Blind human audit of the scorer on 200 responses	Not run (A5); the sheet and sealed key are in the package

Hegre and Sambanis as distributed by Muchlinski et al. (2016) (7,140 country-years); the fit reaches $R^2 = 0.204$ and no column is a candidate, so it is a negative control with no anchor. Neither score reaches the product’s trigger ($R^2 > 0.999$), so on both datasets we call the narrator directly. The prompts, schemas and contract were written against the reference task; the crowded instance and the two datasets played no part in their design. Twelve more instances, frozen in the plan on 2026-07-07 before any live run and first run for this revision, plant one of six registered target-derived leaks (monotone transform, noisy proxy, exact copy, inverse, binned copy, duplicated rows), each in two of four public tables: insurance charges (Lantz 2013) (simulated from census figures), Cleveland heart disease (Janosi et al. 1988), IBM telco churn (IBM 2019) and Ames housing (Cock 2011). In every arm the shipped leakage detector produces E . Section 9 describes the profile task of RQ5.

5.2 Arms

An *arm* is one experimental condition: a fixed system prompt, tool schema and check on the returned answer. Table 3 lists them. The names follow the analysis plan and the replication package, so every number can be traced to its records. The *bare* prompt states the task and no faithfulness rule. The *shipped* prompt, the one mlcompass uses, states seven rules (among them: cite only items in E , copy numbers exactly, address the top candidate) and a verdict mapping, and tells the model that the schema restricts the column field and that violations are rejected; on the arms without enforcement neither is true. An *open* schema has the answer’s fields and lists no admissible values. **A-CONTRACT** runs the shipped enforcement under the bare prompt, so that mechanism is not confounded with instruction. **A-STRESS** removes the Tier A enums so that every Tier B catch is observable. **A-GR-CHOICES** is our reimplement of the toolkit’s acceptable-values validator with its list built from E at call time, because a fresh install of Guardrails AI 0.11 registers no validator. Every arm with a retry loop has a budget of 2. **A-GR-STOCK** and **A-STATIC-ENUM-STALE** are controls, not competitors of equal strength: they are what a practitioner gets by default, a toolkit’s stock validators, which check structure, and a column list written once while authoring. The competitor of equal strength is **A-GR-OURS**, the same checks as Tier B inside the toolkit’s loop.

After review we added controls. The column description of **A-L1-DESCRIBED** reads “Names of dataset columns (features) from the evidence that you cite. Column names only: not metric names such as r^2 , and not statistic names.” That sentence names the failing token, so the **-NEUTRAL** arms end it instead with “Column names only, not the names of metrics or statistics.” **A-L2-RULES** gives the bare prompt followed by the shipped prompt’s first three rules, with no claim that anything is checked. **A-L2-NOCLAIMS** is the shipped prompt with its two sentences claiming that violations are rejected removed and nothing else changed, and **A-L1-T0** repeats the floor at temperature 0.

Table 3 The arms. Each fixes the system prompt, the tool schema and the check on the returned answer. *Open*: the answer’s fields with no admissible values listed. *Call-time enum*: value lists built from E for each request. *Tier B*: our verifier with named retries. Leakage arms run on `deepseek-chat` with $N = 200$; the profile arms of RQ5 (prefix P-) mirror A-L1, A-TIER-A-ONLY, A-CONTRACT, A-STRESS and the two strong arms.

Arm	Prompt	Schema	Check
<i>No check</i>			
A-L1 (floor)	bare	open	none
A-L1-T0	bare	open	none, $T = 0$
A-L2	shipped	open	none
A-L2-NOCLAIMS	shipped, no claim	open	none
A-L2-RULES	bare + three rules	open	none
A-L1-DESCRIBED ^a	bare	open + description	none
A-STRICT-STATIC	bare	typed, no column list	none
A-STATIC-ENUM-STALE	bare	typed + fixed list	none
A-TIER-A-ONLY	bare	call-time enum	none
A-STRICT-ENUM	bare	call-time enum + strict flag	none
<i>Check and retry loop</i>			
A-GR-STOCK	bare	open	Guardrails: structure
A-GR-CHOICES	bare	open	Guardrails: list from E
A-GR-OURS	bare	open	Guardrails + our checks
A-CONTRACT	bare	call-time enum	Tier B
A-L3-SHIPPED	shipped	call-time enum	Tier B
A-STRESS	bare	typed, no enum	Tier B
A-STRESS-GENERIC	bare	typed, no enum	Tier B, generic retry
<i>Well-specified configuration, verifier varied</i>			
A-STRONG-NOVERIFY	shipped	call-time enum + description	none
A-STRONG-CONTRACT	shipped	call-time enum + description	Tier B

^aAlso A-L1-DESCRIBED-NEUTRAL, whose description names no metric, and A-L2-DESCRIBED and A-L2-DESCRIBED-NEUTRAL, which use the shipped prompt.

A-STRONG-NOVERIFY and A-STRONG-CONTRACT give the narrator everything the study found to help (the shipped prompt, the call-time enum and the neutral field description) and differ only in whether the payload is verified. These controls are not part of the registered plan and are exploratory; the confirmatory tests we then registered (amendment A3) were cut short by the provider budget (A4). They ran between 2026-09-24 and 2026-09-30 on later harness commits, which add the options, and each record pins its commit. The baseline arms were also run on the crowded instance, where the author-time enum is stale, and the profile arms of RQ5 were re-run under natural column names.

Sample size and dates Each arm on `deepseek-chat` and `gpt-5.4-mini` has $N = 200$, fixed in the analysis plan; the local `qwen2.5:7b` arms have $N = 20$ and the RQ4 variants $N = 100$. One arm’s responses on one task instance and date form a *cell*, and the arms run together on one date a *battery*. A count of 0 in 200 bounds the rate below 1.9% (Wilson, 95%); power is in Section 12. The provider serves a rolling model name, so the same arm can read differently on

different days. Every rate therefore carries its run date, and arms are compared within a date.

For RQ4 the bare instruction was rewritten six ways (*terse*, *baseline*, *cautious*, *expert*, *helpful*, *mechanical*), with $N = 100$ per variant. None contains a faithfulness rule, and all were written before the September runs. A June pilot measured the same sweep (fault 5, Section 10).

5.3 Measures

We score three structured channels on every response that reaches the user. An *entity* violation is a leaf outside its field’s admissible set; a *value* violation is a claim whose column is in E but whose value misses the measurement by more than τ , or whose pair E does not carry; an *omission* is a committed verdict that never references the anchor. A *response* is one narration as it reaches the user, however many provider calls its retries took; the replication package lists every run file with its responses and calls. We report percentages with Wilson 95% confidence intervals (CIs) and the raw k/N . For an arm with a verifier we report what the verifier did and do not read its zero as a rate. The entity channel decomposes into *invented* and *misfiled*, and the value channel into wrong numbers and pairs E does not carry. We exclude artifacts, namely a `null` column and several real names in one string; counting them as violations changes one response in 13,200. Transport errors are excluded from N and reported.

The tolerance decides almost nothing. Of 62,854 claims scored before any verification could act on them (every unenforced leakage and profile arm, all runs), 60,106 reproduce the measured value exactly and 2,722 round it by less than 5×10^{-4} , so only a tolerance below that would begin to flag rounding. One claim drops a correlation’s sign ($|\Delta| = 0.039$), and the remaining 25 miss by more than 0.05. The number of responses with a value violation is 22 at every τ from 5×10^{-4} to 0.01 and 21 at 0.05 (`scripts/tau_sensitivity.py`). The narrators copy numbers, so the choice matters little here. An absolute τ suits bounded quantities such as correlations; unbounded ones call for a relative tolerance and counts for an exact match, and each specification can set its own. Both of ours use 0.005, which on an integer count rejects every other integer.

`deepseek-chat` is the model on which the failure occurs; `gpt-5.4-mini` and a local `qwen2.5:7b` are the null providers (Section 7.3). Temperature is 1.0, every endpoint’s default and the shipped narrator’s setting, except in `A-L1-T0`. A contract arm with no violation at the default is therefore not an artefact of a lowered temperature, and the baselines’ rates are conditioned on it (at $T = 0$ the floor falls to 2.0%, Section 6).

5.4 Use of AI tools

The authors used Claude (Anthropic), principally Claude Opus 5.5, accessed through Claude Code during 2026. Claude drafted and revised manuscript text in every section from the authors’ notes, results and instructions, including a final editing pass for readability; wrote and tested most of the measurement harness, the analysis scripts that compute the reported tables, and parts of the artifact’s verification code; ran the experiment batteries under the authors’ direction; checked bibliographic entries against their sources; and simulated a multi-reviewer pre-submission review whose findings the authors addressed. The large language models **deepseek-chat**, **gpt-5.4-mini** and **qwen2.5:7b** are subjects of the study and are described above. The authors chose the research questions and the study design, reviewed and edited all text, inspected and tested the code, recomputed every reported number from the released run records, verified every reference, and made all analytical and interpretive decisions. Claude is not an author, and the authors take full responsibility for the content of this paper.

6 RQ1: How the Unconstrained Narrator Fails

On the bare arm **deepseek-chat** cited a name outside the column domain in 42.0% [35.4, 48.9] of 200 responses on 2026-09-15 (84/200). Four runs over nine days read 42.0, 43.0, 40.5 and 49.0%, with no detectable heterogeneity (χ^2 , $p = 0.34$). Decomposed, the rate contains no fabricated column.

Composition of the failures We looked at every arm that runs the bare instruction with no column domain, in every run: sixteen cells of 200 on four task instances, with 1,012 flagged responses. Each of those responses places a real evidence name where it is not admissible. The name is **r2**, which E carries as the name of the suspicious metric, written into the list of cited columns; three times it is **perfect_match_rate**, the name of another quantity in E . One response also cites **target**, a real column of the frame that E does not list (unlisted), and no response invents a name. In the terms of Section 2 the failure is intrinsic. The verifier’s measure is strictly *out-of-evidence*, which is weaker than *non-existent*: it rejects **target** as it would an invented name. The failure is not an artefact of synthetic data: on the two real datasets the bare arm misfiles in 21.5% [16.4, 27.7] (bodyfat) and 20.5% [15.5, 26.6] (sambanis) of responses, every one **r2**, and the shipped prompt and the contract read 0/200 on both.

Twelve further instances On the twelve instances frozen in July the floor arm misfiles in 26.5 to 55.5% of responses per instance, 817 of 1,888 responses in all; 512 calls lost when the provider balance ran out (23 to 64 per instance, all errors 402 or 429) are excluded (amendment A4). Of the 817, 795 misfile **r2** or **perfect_match_rate**. Forty-five cite the frame’s own target column, which

E does not list, 18 of them with no other violation. Twelve cite a name in neither *E* nor the frame: ten write **target** for the target column, and two respell a real one (**charged_est**, **total_charges**). The registered prediction that such names stay under 5% of flagged responses holds (1.5%). The 817 are thus 795 that misfile, 18 that cite only the target column and 4 that cite only an invented name; with the 1,012 of the first four instances, all of which misfile, 1,807 of 1,829 flagged responses misfile. The composition found on four instances holds on twelve that played no part in the design; Table 4 gives each one.

Table 4 The floor arm (A-L1) on the twelve instances frozen on 2026-07-07, **deepseek-chat**, 200 calls each. *Lost*: calls that returned a transport error when the provider balance ran out, excluded from *N* (amendment A4). *Flagged*: responses with an entity violation, with the Wilson 95% interval. The last three columns count responses containing at least one name of that kind, so a response can appear in more than one. Generated by **scripts/table-fabbench12.py**.

Table	Leak	<i>N</i>	Lost	Flagged	% [95% CI]	Misfiled	Unlisted	Invented
Ames	exact_copy	152	48	49	32.2 [25.3, 40.0]	49	1	0
Ames	inverse_target	149	51	49	32.9 [25.9, 40.8]	49	0	0
Ames	monotone_log	159	41	65	40.9 [33.5, 48.6]	58	14	0
heart	binned_target	159	41	75	47.2 [39.6, 54.9]	73	4	3
heart	exact_copy	162	38	60	37.0 [30.0, 44.7]	60	1	0
heart	inverse_target	163	37	84	51.5 [43.9, 59.1]	83	1	3
insurance	contamination	155	45	86	55.5 [47.6, 63.1]	85	3	0
insurance	monotone_log	175	25	85	48.6 [41.3, 55.9]	85	3	1
insurance	noisy_proxy	177	23	91	51.4 [44.1, 58.7]	90	0	3
telco	binned_target	149	51	59	39.6 [32.1, 47.6]	58	4	1
telco	contamination	136	64	36	26.5 [19.8, 34.5]	36	0	0
telco	noisy_proxy	152	48	78	51.3 [43.4, 59.1]	69	14	1
All		1,888	512	817		795	45	12

Cause: an undescribed field The bare instruction says “list every column from the evidence you cite”, and the open schema says nothing about the cited-column field. The suspicious metric is the thing under investigation, so the model cites it. One sentence of field description takes the rate from 49.0% (98/200, A-L1 on 2026-09-24) to 0.5% [0.1, 2.8] (1/200), whether the sentence names **r2** (A-L1-DESCRIBED, same day) or only says “not the names of metrics or statistics” (A-L1-DESCRIBED-NEUTRAL, next day). Rules do less on their own. The shipped prompt’s first three rules, stated without enforcement claims, leave 7.0% [4.2, 11.4] (14/200, A-L2-RULES), every one **r2**. The full shipped prompt gives 0% (0/200, below 1.9% at 95% confidence; A-L2), which the analysis plan predicted when it chose the bare prompt for the comparison (Table 2). Removing only its two sentences claiming that violations are rejected leaves it at 0% (0/200, A-L2-NOCLAIMS, 2026-09-27), so its advantage over the three rules comes from the rest of its content: four further rules, a verdict mapping and its emphasis. Our reference names do not explain the misfiling: on

the crowded instance, whose columns all have different names, the undescribed field gives 61.0 % [54.1, 67.5] (122/200), every one `r2`. Nor does it occur on every model. `gpt-5.4-mini` never writes `r2` into a column field in 200 responses, and a local `qwen2.5:7b` never does so in 20.

Temperature Rerun at $T = 0$ on 2026-09-27, the bare arm misfiles in 2.0 % [0.8, 5.0] of responses (4/200, `A-L1-T0`), every one `r2`. The failure is rarer but not gone, and the temperature is one more setting chosen by whoever calls the endpoint.

Consequences On this model the characteristic failure is the evidence’s own vocabulary leaking into a field whose admissible set nobody stated. It remains a violation: a consumer that drops, correlates or flags `r2` as a leak candidate acts on something that is not a column. Every arm faced the same instruction, so the arm comparisons stand.

Abstention A narrator that retreated into “cannot determine” would score zero on every channel for free. Of the 46 abstentions on instances with a leak, 41 are on the crowded instance’s stale-enum arm, whose list contains none of the evidence’s columns, and `gpt-5.4-mini` never abstains. A further 13 responses carry an empty verdict (8 of 160 on `qwen2.5:7b`, 5 on `deepseek-chat`), all on arms without a verifier. The zeros of the enforced arms therefore come from narrators that committed to a verdict.

Answer to RQ1 The failing model writes the evidence’s own vocabulary, above all `r2`, into an undescribed column field; 12 of 1,829 flagged responses name something the data lacks. A field description, the shipped prompt or temperature 0 lowers the rate from 42–49 % to 0–2 %.

7 RQ2 and RQ3: Enforcement, Its Cost, and the Alternatives

7.1 Comparing the mechanisms

Table 5 gives the rates. On the reference task, every arm that states the admissible column set reaches zero or one violation in 200, whether it states the set in the shipped prompt, a description on the field, an enum in the schema, a choice list in a guardrail loop, or our verifier. The one exception is the shipped prompt’s first three rules on their own, at 7.0 %. A typed schema without a column enum reads 4.0 %, and the arms that state nothing read 35–49 % across runs. The cleanest comparison is the same-date one inside a single toolkit. On 2026-09-18, `A-GR-STOCK` and `A-GR-CHOICES` ran the same Guard, reask loop, budget, prompt and model, and read 43.5 % (87/200) against 0 % (0/200, with 88 responses sent back and repaired). The stock configuration validates structure and has no faithfulness validator, because a fresh install

Table 5 Percentage of responses reaching the user with a structured violation, 200 per cell, `deepseek-chat`. *Date* is the 2026 run date of the reference cell; the crowded instance, whose evidence columns the author-time enum does not cover, ran on 2026-09-24/25. A sampled 0 bounds the rate below 1.9 % (Wilson, 95 %). For the arms with a verifier (below the second rule) the zero holds by construction, and Table 6 reports what the verifier did. Table 3 defines the arms.

Arm	Mechanism	Date	Reference (%)	Crowded (%)
A-L1	no enforcement	09-15	42.0	61.0
A-L1-T0	no enforcement, $T = 0$	09-27	2.0	—
A-GR-STOCK	Guardrails, no faithfulness check	09-15	35.0	57.5
A-STRICT-STATIC	schema, no column enum	09-15	4.0	27.5
A-STATIC-ENUM-STALE	author-time column enum	09-15	0 ^a	72.5 ^b
A-L2	shipped prompt	09-24	0	—
A-L2-NOCLAIMS	shipped, no enforcement claim	09-27	0	—
A-L2-RULES	its first three rules only	09-25	7.0	—
A-L1-DESCRIBED	description naming <code>r2</code>	09-24	0.5	—
A-L1-DESCRIBED-NEUTRAL	description, no metric named	09-25	0.5	—
A-TIER-A-ONLY	call-time enum	09-16	0	0
A-STRICT-ENUM	call-time enum, strict flag	09-24	0	—
A-GR-CHOICES	Guardrails, choices bound to E	09-18 ^c	0	1.5 ^d
A-GR-OURS	Guardrails, our checks	09-15	0	0
A-CONTRACT	Tier A + Tier B	09-15	0	0
A-STRESS	Tier B alone	09-15	0	0
A-L3-SHIPPED	as shipped, prompt included	09-24	0	—
A-STRONG-NOVERIFY	shipped + enum + description	09-30	0	0
A-STRONG-CONTRACT	the same + Tier B	09-30	0	0

^aOn the reference task the author-time list equals the evidence’s ten columns, so the arm is identical to the call-time enum. ^b145 of 200, all invented; 132 of them also omit the anchor, which the list does not contain, and 41 abstain. ^cA-GR-STOCK read 43.5 that day.

^d3 of 200, with one misfiled name in a claim’s column field and two misquoted values, in the fields a single choice list does not check.

registers none, so nothing in it could reject `r2`. The choice arm is our reimplementa-
tion of the toolkit’s acceptable-values semantics with its list built from E at call time. It checks the cited-column list and nothing else. On the reference task it matched our validator (A-GR-OURS, where no violation reached the user and 84 responses were repaired). On the crowded instance it let three responses through, one with a misfiled name in a claim’s column field and two with misquoted values, while our validator in the same loop let none through (118 repairs). The binding does the work on the field it covers, and Tier B’s other checks cover the fields a choice list leaves open.

A-STRICT-STATIC, a schema with types and required fields and no column enum, reads 4.0–7.0 % across runs against the floor’s 42 %. A schema therefore steers a model strongly even where it constrains nothing, which contradicts the prediction of H8 (a rate within 2 points of the floor) without meeting its registered rejection rule, 0/200 (Table 2). The evidence-bound enum without verification left no violation on the reference task with or without the provider’s strict flag, so hypothesis H9 holds on the entity channel. Neither result amounts to a guarantee. In the non-strict arms the enum ran as a request, and whether

a strict flag is honoured is up to the provider. None of the 400 responses sent with the strict flag carried a column outside the enum, but from outside the provider a model that obeyed and a server that quietly replaced a value look the same.

Against a well-specified configuration Each alternative above lacks something. The two strong arms give the narrator the shipped prompt, the call-time enum and the neutral field description, and differ only in whether the payload is verified. On 2026-09-30 both read 0/200 on the reference, the crowded instance and the two real datasets; on the twelve frozen instances the budget left 50 unverified responses, none with a violation, and no verified ones (A4). The verifier never had to send a response back, and median latencies were within 0.1 s of each other (2.4–2.7 s). In a configuration this well specified we observed no gain in rate on leakage. The profile task of RQ5 gave the one difference we observed. Under natural column names the unverified strong arm delivered a violation in 4 of 200 responses (2.0 % [0.8, 5.0]): one cites `reference_count` beside the real `referral_count`, two attach real numbers to invented `*_placeholder` columns, and one to a statistic *E* lacks. The verified arm, an independent sample, delivered none; it sent back four of its own responses, three citing columns the frame lacks and one without an admissible verdict, and one retry repaired each. Against 0/200 this is not significant (Fisher $p = 0.12$) and was found in an exploratory run. The confirmatory test we then registered, 600 responses per arm (amendment A3), stopped when the provider balance ran out after 93 and 91, none with a violation, and we run no test on them. What the verifier adds is a check whose result does not depend on the configuration, at no cost we could measure.

An author-time enum on new data On the reference task the author-time list is the evidence’s own column set, which makes the stale-enum arm the call-time arm under another name. The two diverge on the crowded instance. There the list written against the reference task covers none of the ten evidence columns, and the model obeys it. It cites reference-task columns that do not exist in this frame in 145 of 200 responses (72.5 % [65.9, 78.2]). Every one of these is an invention, a failure the undescribed field never produced. In addition, 139 responses omit the anchor, which the list does not allow them to name (132 of the 145, and 7 that commit while citing nothing), and 41 abstain. The same instance reads 27.5 % (55/200) with no column enum and 0 % (0/200) with the enum built at call time. On evidence the list predates, the author-time list produces the failure it was supposed to prevent.

7.2 Cost of verification

Table 6 shows what an enforced arm’s zero consists of. One named retry repaired every violation the verifier met; only the content-free correction of **A-STRESS-GENERIC** ever needed a second, twice in 200. Nothing was stripped

Table 6 What the verifier did on the enforced arms (reference battery and the profile task), $N = 200$ each. A response is *clean* when its first attempt satisfied (C1)–(C2); otherwise one named retry repaired it. No response on these arms needed a second retry, and none was stripped. **A-GR-CHOICES** ran on 2026-09-18. Calls and wall-clock latency are per response; **A-L1** and **P-L1** are the unenforced baselines.

Arm	clean	retried	calls	latency
A-L1	—	—	1.00	2.8 s
A-CONTRACT	200	0	1.00	3.3 s
A-STRESS	188	12	1.06	3.6 s
A-GR-OURS	116	84	1.42	4.5 s
A-GR-CHOICES	112	88	1.44	4.9 s
P-L1	—	—	1.00	5.0 s
P-CONTRACT	194	6	1.03	5.5 s
P-STRESS	187	13	1.07	5.0 s

anywhere in the study, so the deletion that guarantees (C1)–(C2) remained a worst case that never occurred. With nothing to repair, the contract arm took 3.3 s per response against 2.8 s for the floor (medians 3.3 and 2.8 s, 95th percentiles 3.9 and 3.3 s). The check is not what costs: offline, verification and stripping take a median of $18\mu\text{s}$ per response, and **A-TIER-A-ONLY**, which sends the same enum schema without verifying, took 3.5 s on another date. The difference is therefore not the check; the larger schema is the likely cause, although runs on different dates cannot separate it from provider variation. Arms whose responses need retries cost more on average: 3.6 s per response on **A-STRESS** and 4.5–4.9 s on the Guardrails arms. Removing the enum while keeping the loop and verifier raises the number of retried responses from 0 to 12 on leakage and from 6 to 13 on the profile task. The schema does not strengthen the guarantee; it reduces retries on the channel it can express. Retry counts vary between runs (Section 12), so the cost has to be reported per run; the guarantee does not depend on it. In neither table: **A-L2-DESCRIBED** and **A-L2-DESCRIBED-NEUTRAL** read 0/200, the shipped contract under the RQ4 wording that fails most often returned no violation, and on the crowded instance **A-STRESS**, **A-GR-CHOICES** and **A-GR-OURS** retried 25, 101 and 118 of 200 responses. The contract path does not surface token counts.

Why Tier B is still needed Our schemas bind each field to an independent **enum**. That form expresses the column domain, which is why the enum arms tie the contract on leakage, but not the other two channels: a value’s admissible set is a tolerance band that depends on the column and statistic a sibling field chose, and “a committed verdict addresses the anchor” is a property of the whole response. Full JSON Schema could express both for a finite E , with one **oneOf** branch per (column, statistic) pair carrying **minimum** and **maximum**, and **if/then** with **contains** for the anchor. Provider strict modes accept subsets of JSON Schema, such a schema grows with $|V_E|$, and we did not test one. Whatever the schema, its enforcement happens inside the provider; Tier B checks the returned payload in our own process. Where logits are available,

Table 7 Rule-free instruction variants, $N = 100$ each, `deepseek-chat`, ordered by the blended entity rate, the share of responses with any entity violation. *Outside E* is a name *E* does not carry (18 of the 20 are **target**, the frame’s real target column, which *E* omits); *misf.* is a name in *E* that is not a column; *misq.* is a wrong number on a pair *E* carries; *unver.* is a pair *E* does not carry. *Statistic specified* is what the instruction says about the claim’s statistic.

Variant	Statistic specified	entity channel				value	
		blended	95 % CI	outside E	misf.	misq.	unver.
terse	" correlation " named	9.0 %	[4.8, 16.2]	7	4	0	1
baseline	" correlation " named	36.0 %	[27.3, 45.8]	0	36	0	0
expert	nothing	56.0 %	[46.2, 65.3]	5	51	0	45
cautious	"each correlation"	68.0 %	[58.3, 76.3]	2	68	1	8
helpful	"every correlation number"	71.0 %	[61.5, 79.0]	5	71	0	10
mechanical	the field, not its content	72.0 %	[62.5, 79.9]	1	71	0	26

grammar-constrained decoding could enforce (C1) by construction and, with a grammar generated for each call, encode more of (C2) and (C3). We did not run that comparison: the one open model we ran, `qwen2.5:7b`, did not fail without enforcement, so there was nothing for it to prevent.

7.3 Providers without failures

On `gpt-5.4-mini` (same evidence, prompts, scorer and budget, and no transport errors) all seven arms that existed at the time show no violation in 200, the floor included. That model cites exactly the ten evidence columns in 199 of 200 responses and never writes `r2` into a column field. A local `qwen2.5:7b` ($N = 20$) shows none except `A-STRICT-STATIC` at 10 % (2/20), both the invented name `suspicious_metric.name`. Those providers offer nothing to compare, so the comparative claims of this section rest on one model, and its rates describe `deepseek-chat` on particular dates, not LLMs in general.

Answer to RQ2 and RQ3 Under the contract no structured violation reached the user; one named retry repaired each, and the check itself costs microseconds. Added to a well-specified configuration it changed no rate on leakage, and on profiling it caught 4 of 200 in an exploratory run. A stock guardrail loop stays near the floor, an enum written in advance causes invention on new data, and an enum or choice list bound to *E* matches the contract only on the column field.

8 RQ4: Wording and Specification Move Compliance

Wording and specification Table 7 gives the six variants. Six rule-free variants of the bare instruction, over identical evidence and schema, move the blended entity rate from 9.0 % to 72.0 %. It is already established that wording moves behaviour this much (Sclar et al. 2024). The variants also differ in how much of the answer they specify, and that ordering predicts the unverifiable channel

exactly. It reads 0 and 1 per hundred where the instruction names the statistic, 8 and 10 where it mentions correlations, 26 where it names the field without saying what goes in it, and 45 where it says nothing. On the entity channel, the baseline instruction’s “list every column from the evidence you cite” produces pure misfiling, and Section 6 shows that describing the field removes it.

Out-of-evidence names Of the 20 names in 600 responses that *E* does not carry, 18 are **target**, a real column of the frame that *E* omits. Only two are genuine inventions, both from *expert*, and misquotation of a carried number occurs once. Across variants the out-of-evidence and misfiled counts show no detectable association (Spearman $\rho = -0.25$, $p = 0.63$), though with six variants the test has almost no power. *Terse* has the lowest blended rate and the most out-of-evidence names, but 7 against *mechanical*’s 1 is not significant ($p = 0.065$).

Interpretation Measured compliance belongs to a configuration: the wording, what the schema says about each field, the names in the data, the provider and the date. Two diagnostic arms show the same from inside the contract. The evidence-bound enum alone, with no verification, left no violation in 200 (A-TIER-A-ONLY), and the full contract under the variant that fails most often without enforcement produced no violation on any first attempt.

Answer to RQ4 The wording, and how much it specifies, moves the rate from 9 to 72%, and how much of the answer an instruction specifies predicts the unverifiable channel. The effect of a wording is known only for the configuration it was measured in.

9 RQ5: Does the Contract Transpose?

The second narrated artifact is a deterministic dataset profiler that ships in the same tool. It reports per-column type, missing counts and rates, descriptive statistics, cardinality and outlier counts, plus a task-level class balance. We bind it through the contract layer of Section 4.2, with no branch that names either task. The shape differs from leakage: the entity domain is every column of the frame (24 against 10), the statistic domain has 13 names against 1 and depends on the column, and *E* carries 196 measured quantities against 10. The evidence comes from running the profiler on a frame with ordinary data-quality problems: mixed types, missingness that rises across the categorical block, and injected outliers. That frame named its columns **num_feature_k** and **cat_feature_k**. We also ran the unverified and strong arms on the same data under distinct natural names (**age**, **region**, ...) that share no suffix.

Wrong numbers and column naming Table 8 gives the counts. On the suffixed frame the unconstrained arm put a wrong number in 9 of 200 responses (4.5% [2.4, 8.3]). Eleven of its 12 wrong numbers are a real measurement of the column that shares the suffix, such as **cat_feature_07**’s missing rate reported

Table 8 The second task, $N = 200$ per arm, **deepseek-chat**. A *wrong number* is a claim on an admissible (column, statistic) pair whose value misses the measurement, counted in responses of 200. A *misplaced* claim quotes a number E carries, correctly, under a pair the contract does not admit. Tier B catches are the responses the verifier sent back.

Arm	Names	wrong no.	misplaced	entity	any, % [95 % CI]	Tier B
P-L1	suffixed	9	9	0	8.5 [5.4, 13.2]	—
P-TIER-A-ONLY	suffixed	11	1	0	6.0 [3.5, 10.2]	—
P-CONTRACT	suffixed	0	0	0	0	6
P-STRESS	suffixed	0	0	0	0	13
P-L1	natural	0	26	1	13.0 [9.0, 18.4]	—
P-TIER-A-ONLY	natural	0	4	2	3.0 [1.4, 6.4]	—
P-STRONG-NOVERIFY	natural	0	3	1	2.0 [0.8, 5.0]	—
P-STRONG-CONTRACT	natural	0	0	0	0	4
P-STRONG-*	suffixed	0	0	0	0	0

for `num_feature_07`, and one appears nowhere in E . Under distinct names the same model, on the same data, puts a wrong number in none of 400 unverified responses. The failure the original frame measured came from names our generator chose.

Correct quantities in inadmissible slots Every other value violation, on both frames, quotes a number E really carries, and quotes it correctly. Most are the class balance, which **gpt-5.4-mini** also misplaces in 3 of 50 probe responses. The profiler reports it at task level, the narrator attaches it to the target column as `class_balance`, and the contract admits only per-column statistics, so the correct figure has no admissible slot. Counting these as an invented quantity kind, on the belief that the profiler computes no class balance, was fault 13 in Table 9. Under Definition 2 such a claim is misfiled, a real item placed where it is not admissible, and the verifier rejects it. A reader loses nothing from it, whereas a program that consumes the claim slot receives a wrongly keyed value. Excluding it is a design choice of ours. Re-scored with the class shares admitted on the target column, the unconstrained arm falls from 26 to 10 violating responses under natural names and from 17 to 10 under suffixed ones, and the three **gpt-5.4-mini** violations disappear. What remains is the wrong numbers of suffixed names, other real quantities such as the frame’s shape attached to a column, and one invented name.

How the tiers separate Binding the enum removes misplaced claims almost entirely (26 to 4 under natural names) and leaves wrong numbers in place (9 against 11 under suffixed ones). An enum can refuse a pair outside the admissible statistics, but it cannot constrain a number whose admissible set is a band around a value chosen by a sibling field. Tier B handles both. **P-CONTRACT** returned no violation and sent back 6 responses, and **P-STRESS**, with the enum removed, sent back 13. A re-run of **P-CONTRACT** that recorded every attempt (200 responses, 8 catches) found no entity violation on any first attempt. The enum therefore held on 399 of 400 first attempts there, and we

have no explanation for the one exception. It did not hold everywhere: without the strict flag, 2 of 200 natural-name P-TIER-A-ONLY responses wrote statistic names into the column field. The re-run also caught an interquartile-range outlier count claimed for `cat_feature_03`: the column and the statistic are each admissible, but *E* does not carry the pair; a per-field enum cannot express that relation, and (C2)’s domain check does.

Scope of the result The contract binds to a second evidence shape without task-specific verification, and the two tiers separate once the failing channel is one the per-field enums we use cannot express. The result does not show that misfiling numbers is a general property of narrators. On this model it followed our naming, and what survived distinct names reflects our binding choices as much as model error.

Answer to RQ5 The contract binds to a second evidence shape without task-specific code. There Tier B catches wrong numbers that the per-field enums we use cannot express, and the wrong numbers themselves came from our column naming.

10 Corrections to Our Own Instruments

Every number in this paper comes from an instrument we wrote. Table 9 lists the fifteen faults we found in those instruments, how we found each and what it moved; the replication package keeps the superseded measurements beside their corrections. Faults 1, 3, 6 and 7 concern the downstream benchmark (Section 11). Fault 14 was caught before its numbers were reported and changed what the free-text measurement counts. Fault 15 was found by a second scorer, written from the definitions and sharing no code with the first, which scored all 16,698 leakage records. The two agree on every entity, misfiled and out-of-evidence flag. They disagree on 10 wrong-number flags, each a derived quantity such as a correlation gap, which the first scorer rightly files as a pair *E* does not carry, and on 7 responses on the stale-enum arm that commit to a verdict and cite nothing: the definition counts them as omitting the anchor, and the first scorer did not. Both scorers come from the same definitions, so their agreement speaks to the implementation, not to the definitions.

Three of the faults show patterns the others repeat. In faults 2, 12 and 13 a scorer reported the narrator wrong for being right: it compared every claim with its column’s correlation whatever statistic the claim named, compared `abs_corr` with a signed value, and called a class balance invented although the profiler reports one. We found fault 2 because two readers disagreed about what a channel measured. Fault 9 is a control that silently received the treatment. The profile floor ran the deletion step unconditionally and read 0% on a task that fails, and we noticed only because the arm printed sixty Tier B catches although it has no Tier B. The corrections also had a direction. The

Table 9 Instrument faults, numbered in the order found.

#	Fault	Found by	Moved
1	checklist penalised unlisted correct forms	authors	defects 40/31/15 → 31/24/8
2	value scorer ignored the statistic field	two readers	sweep value flags 85→2
3	the fix for 1 matched key- word arguments	authors	within 1
4	provider pin refused forced tool calls	authors	one battery 0/0 →usable
5	June pilot (11.5 %, sweep 1– 100 %) did not reproduce; rolling alias, records not kept	authors	pilot not re- ported as a result
6	target harvest missed pa- rameter defaults	second scorer	control 33→31
7	runner executed cells off the frozen plan	authors	48 runs rela- belled
8	narration never recorded	authors	free text re- run with it
9	profile floor stripped before scoring	its telemetry	P-L1 0→17
10	statistic names ours, not the profiler's	authors	$\approx \frac{1}{3}$ of first rate
11	cost table transcribed by hand	authors	table regener- ated
12	magnitude aliases scored as signed	review	sweep value flags 2→1
13	correct class-balance claims called invented	re-run	P-L1 wrong- number 17→9
14	free-text scorer: prompt names, wrong E for crowded, thousands separa- tors	re-run	prompt arms 91–98→6– 22 %
15	a verdict citing nothing not counted as committing	second scorer	stale-arm omissions 132→139

first four that moved the downstream checklist all moved it toward our hypothesis, because false positives concentrate in careful code and the treatment arms write more of it. The two that broke the pattern came from procedures designed to look for it: a second scorer written with a different technique, which agreed with the first on 856 of 858 rule instances before fault 6 was fixed and on all 858 afterwards, and an arm whose adverse outcome we wrote down before it ran.

We also found defects in the artifact itself, which the table does not count because the artifact is not an instrument. A claim value of NaN passed the verifier, because every comparison with NaN is false. The property-based falsifier that the analysis plan registered, which we built late, reproduces that refutation at its third generated case on the earlier code. It then found a second defect: a 400-digit integer, which a JSON parser returns as an exact integer, crashed the value check. Both are fixed, and the fixed verifier passes 10^6 generated cases. No recorded response carries either value, so no reported number changes.

11 Discussion

Cheap fixes and their reach The misfiling of Section 6 disappears with the shipped prompt, with or without its claim that violations are rejected, or with one sentence of field description; the prompt’s rules alone leave 7.0 % and temperature 0 leaves 2.0 %. The wrong numbers of Section 9 disappear under distinct column names, and the whole failure on another provider. Each comes back where its fix is absent: 61 % on a differently named instance, 20–56 % on the fourteen public-data instances, 49 % on the same model nine days later. A tool author controls the prompt and the schema, not the names in the user’s data, the provider behind an endpoint or the date, and cannot know that every field is specified well enough for every model without measuring each combination. A fix’s effect is known only where it was measured; the verifier’s follows from code that runs in our process.

Binding at call time An enum written for the reference task turned 61 % misfiling into 72.5 % invention on the crowded instance, none of whose columns it lists; the call-time enum read zero. That control is extreme, and partly stale lists, the realistic case, are untested.

Simpler designs If every structured fact were rendered deterministically and the model wrote only prose, the structured guarantee would be trivial and the model would be left with the one channel no check reaches. Stripping without retry is cheaper, but it would have deleted content from 6–16 % of A-STRESS responses and 41.5–59 % of the Guardrails arms’ responses, all of which a single named retry repaired (Table 6). Applied offline to the 2,089 violating responses of every unverified leakage arm, deletion would keep 87.5 % of their cited columns and 77.0 % of their claims, never lose the anchor, and leave

554 with no claim. Nearly every claim it removes (3,865 of 3,888) quotes a number E carries, mostly under a statistic name the domain does not admit, so deletion costs correct information, and the retry is what avoids that.

A downstream benchmark A pre-registered benchmark of whether the tool’s findings improve the training scripts a model writes is in the replication package with its protocol, runs and both scorers. It measures the tool’s advice, not the narration contract, and it cannot show a benefit even of the advice: a control arm told only the six scored defects ended with fewer of them than the arm given the tool’s findings, while producing a runnable script in 47 % of runs against 78–83 % for the other arms. We draw no claim from it.

Where the contract transposes The contract needs a deterministic producer, a finite set of observed entities and values, and a runtime domain bound to that set with independent verification. Narrators of static-analysis findings, continuous-integration failures or dependency-scanner reports meet these conditions, and there too the guarantee would be consistency with the tool’s output, not correctness.

12 Threats to Validity

Construct The guaranteed property is referential and value soundness on the structured channels. The verdict, the prose and the correctness of E lie outside it (Section 4). We measure faithfulness to E , not its effect on an engineer’s decisions. The free text could in principle absorb what enforcement removes from the structured fields. Every run on the two synthetic instances since the record normaliser was fixed (fault 8) keeps the narration: 7,198 narrations on `deepseek-chat` in 36 cells. We checked every number in each against every quantity in E , and every identifier against every name in E and in the system prompt. On the reference task, arms that can reject a claim carry ungrounded content in 8.8 % [7.6, 10.2] of narrations and arms that cannot in 12.6 % [11.5, 13.8]. Within the rejecting arms, responses that went through a retry read 10.9 % [7.7, 15.4] against 8.4 % [7.1, 9.9] for the rest (Fisher $p = 0.19$). On the crowded instance the rejecting arms read 5.9 % [4.7, 7.4] and the others 4.5 % [3.4, 6.0], leaving aside the stale-enum arm, whose narrations repeat the invented names of its structured fields (58.8 %); retried responses read 8.6 % against 5.2 % ($p = 0.07$). The retried responses lean the way displacement would predict, by 2.5 and 3.4 points, which we cannot tell from noise at this size. We therefore claim only that the scan found no clear increase, within what it can see. The instrument has one blind spot: a misfiled name is in E and so counts as grounded in prose, so misfiling displaced into the narration would not register. No enforced arm stripped anything, which means the pressure we tested is correction and not deletion, and unsupported causal prose would need human raters. Run at call time as a warning, the same matcher would fire on 5–13 % of narrations, mostly on true bounds such as “below 0.05” and

shortened names; mlcompass does not ship it. The value channel was stressed less than the entity channel: the registered value-pressure instance (H14) did not activate it, and (C2) caught real errors only on the profile task.

Internal We compare arms within one date. Arms ran in blocks, not interleaved, so drift within a day is not controlled. The provider serves a rolling model name, and the unconstrained rates move between runs: **A-GR-STOCK** read 35.0, 43.5 and 47.5 % over nine days (heterogeneity $p = 0.035$), and **A-STRESS** repaired 12, 23, 32 and 19 responses in four runs ($p = 0.012$), with 32 against 19 on the same date ($p = 0.07$), so the date does not account for all of the variation. The harness and adapter also changed between runs, and the provenance of every record pins both, so a difference across dates cannot be attributed to the provider alone. **A-L1** did not move ($p = 0.34$).

External The misfiling occurs on one of three providers, so every comparative claim rests on a single provider, and every rate is conditioned on **deepseek-chat**, the temperature (the floor reads 49.0 % at $T = 1.0$ and 2.0 % at $T = 0$) and the date. Two leakage instances are synthetic and two are real datasets, one with a documented leak and one without, and twelve plant registered leaks in four public tables, where only the floor arm completed. The profile frame is synthetic data under two naming schemes. We registered a measurement of the shipped default narrator, **claude-opus-4-7**, and did not run it for lack of budget. We did measure schema cost: from 10 to 1,000 columns the enum costs 38 bytes per column, against 344 for the evidence that produced it. Provider limits on enum size are unmeasured, and the retry, which inlines the list, grows with the domain.

Conclusion validity At $N = 200$ a comparison against a zero arm reaches $p < 0.05$ at six events and detects a true rate of 4 % with power 0.8 (Fisher). The resolution is coarser elsewhere: near the 42 % floor, two arms must differ by about 13 points for the same power, so mechanisms closer than that cannot be separated there. Only the registered family is corrected for multiplicity (Table 2); every other interval is descriptive.

13 Conclusion

On the one model of three that fails, an unconstrained narrator of structured evidence misfiles the evidence’s own vocabulary into an under-specified field; across sixteen leakage instances, 12 of 1,829 flagged responses name something the data lacks. The shipped prompt, a field description, distinct names or another provider each remove the failure, temperature 0 nearly does, and it returns wherever the fix is absent. For narration whose citable entities and quantities are finite and enumerable at call time, binding each field’s admissible set to the evidence and checking the returned payload in host code, which is ordinary validate-and-reask, enforces referential and value soundness behind

an API that exposes no logits. The check held in every configuration in which it ran and repaired every violation with one named retry; the check itself takes microseconds. Added to a well-specified configuration it changed no rate on leakage and caught 4 of 200 responses on profiling, an exploratory difference whose registered confirmatory test the budget cut short. We report fifteen faults in our own instruments and two in the verifier, with what each moved. The check does not yet reach a client model that narrates the same evidence across the tool-server boundary.

Acknowledgements We thank the readers of earlier drafts whose disagreement located the value scorer’s defect.

Statements and Declarations

Funding. No funding was received for conducting this study.

Competing interests. The authors have no competing interests to declare that are relevant to the content of this article. mlcompass is released by the authors under the MIT licence and is not a commercial product.

Ethics approval. Not applicable: the study involved no human participants and no animals.

Consent to participate and to publish. Not applicable.

Author contributions. H.S. designed and implemented the contract and the measurement harnesses, and ran the batteries. Y.Ü. designed the downstream benchmark protocol and performed the independent review that identified scorer false positives. M.K.Ö. supervised the work. All authors read and approved the final manuscript.

Data availability. The replication package (Sabuniş et al. 2026) contains the artifact source, every harness, the frozen analysis plan and its amendments, one record per live response with the verbatim model reply, the scripts that recompute every table from those records, the registered property-based falsifier, the superseded measurements beside their corrections, an inventory of every run file from which each count in this paper is recomputed, and the downstream benchmark. Code is MIT; data is CC-BY 4.0. Most of the harness and analysis code was drafted with an AI assistant and then inspected and tested by the authors.

Code availability. The source of mlcompass, every harness and every analysis script are in the replication package (Sabuniş et al. 2026) under the MIT licence; the development repository is <https://github.com/hakansabunis/mlcompass>.

Use of AI tools. Documented in Section 5.4.

References

- Abadi M, Barham P, Chen J, Chen Z, Davis A, Dean J, Devin M, Ghemawat S, Irving G, Isard M, Kudlur M, Levenberg J, Monga R, Moore S, Murray DG, Steiner B, Tucker P, Vasudevan V, Warden P, Wicke M, Yu Y, Zheng X (2016) TensorFlow: A system for large-scale machine learning. In: Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pp 265–283, 1605.08695
- Agrawal LA, Kanade A, Goyal N, Lahiri SK, Rajamani SK (2023) Monitor-guided decoding of code LMs with static analysis of repository context. In: Advances in Neural Information Processing Systems (NeurIPS)
- Ahn J, Kim M (2026) From prompts to contracts: Harness engineering for auditable enterprise LLM agents. arXiv:2607.08028, arXiv preprint; accessed 2026-09-18, 2607.08028
- Anthropic (2024) Model context protocol specification. <https://modelcontextprotocol.io>
- Beurer-Kellner L, Fischer M, Vechev M (2023) Prompting is programming: A query language for large language models. Proceedings of the ACM on Programming Languages 7(PLDI):1946–1969, DOI 10.1145/3591300
- Chen M, Tworek J, Jun H, Yuan Q, et al. (2021) Evaluating large language models trained on code. arXiv:2107.03374, 2107.03374
- Chen R (2025) Evidence-bound autonomous research (EviBound): A governance framework for eliminating false claims. arXiv:2511.05524, 2511.05524
- Cock DD (2011) Ames, Iowa: Alternative to the Boston housing data as an end of semester regression project. Journal of Statistics Education 19(3), DOI 10.1080/10691898.2011.11889627
- Cruz C (2026) ATLAS-RTC: Closing the loop on LLM agent output with token-level runtime control. arXiv:2603.27905, arXiv preprint; accessed 2026-09-18, 2603.27905
- Dhingra B, Faruqui M, Parikh A, Chang MW, Das D, Cohen W (2019) Handling divergent reference texts when evaluating table-to-text generation. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), DOI 10.18653/v1/P19-1483
- Du E, Zhou H, Du C, Liu S, Chen Z, Zheng Z, Zhang Y (2026) LedgerMind: Provenance-constrained multimodal agentic reasoning with a structured evidence ledger. arXiv:2607.28374, arXiv preprint; accessed 2026-09-18, 2607.28374
- Dušek O, Novikova J, Rieser V (2020) Evaluating the state-of-the-art of end-to-end natural language generation: The E2E NLG challenge. Computer Speech & Language 59:123–156
- Falcone Y, Fernandez JC, Mounier L (2012) What can you verify and enforce at runtime? International Journal on Software Tools for Technology Transfer 14(3):349–382
- Feurer M, Klein A, Eggenberger K, Springenberg JT, Blum M, Hutter F (2015) Efficient and robust automated machine learning. In: Advances in Neural Information Processing Systems (NeurIPS), pp 2962–2970, URL <https://papers.nips.cc/paper/5872-efficient-and-robust-automated-machine-learning.pdf>
- Geng S, Josifoski M, Peyrard M, West R (2023) Grammar-constrained decoding for structured NLP tasks without finetuning. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 10932–10952, DOI 10.18653/v1/2023.emnlp-main.674
- Geng S, Cooper H, Moskal M, Jenkins S, Berman J, Ranchin N, West R, Horvitz E, Nori H (2025) JSONSchemaBench: A rigorous benchmark of structured outputs for language models. arXiv:2501.10868, arXiv preprint; accessed 2026-09-27, 2501.10868
- Greshake K, Abdelnabi S, Mishra S, Endres C, Holz T, Fritz M (2023) Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), pp 79–90

- Guardrails AI (2026) Guardrails AI: Adding guardrails to large language models. <https://github.com/guardrails-ai/guardrails>, software repository. Accessed 14 September 2026
- Huang L, Yu W, Ma W, Zhong W, Feng Z, Wang H, Chen Q, Peng W, Feng X, Qin B, Liu T (2023) A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv:2311.05232, 2311.05232
- IBM (2019) Telco customer churn (sample data set). <https://github.com/IBM/telco-customer-churn-on-icp4d>
- Janosi A, Steinbrunn W, Pfisterer M, Detrano R (1988) Heart disease. UCI Machine Learning Repository, doi: 10.24432/C52P4X
- Ji Z, Lee N, Frieske R, Yu T, Su D, Xu Y, Ishii E, Bang YJ, Madotto A, Fung P (2023) Survey of hallucination in natural language generation. *ACM Computing Surveys* 55(12):1–38, DOI 10.1145/3571730
- Jimenez CE, Yang J, Wettig A, Yao S, Pei K, Press O, Narasimhan K (2024) SWE-bench: Can language models resolve real-world GitHub issues? In: International Conference on Learning Representations (ICLR), 2310.06770
- Karnatak N, Chatila M, Hernández DAP, Yazdanfar R, Dugas M, Vakis R (2026) Learning from AVA: Early lessons from a curated and trustworthy generative AI for policy and development research. arXiv:2604.17843, arXiv preprint; accessed 2026-09-27, 2604.17843
- Kaufman S, Rosset S, Perlich C, Stitelman O (2012) Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data* 6(4):1–21, DOI 10.1145/2382577.2382579
- Lantz B (2013) Machine Learning with R. Packt Publishing, Birmingham, UK
- Le Y (2026) Schema-key wording as an instruction channel in structured generation under constrained decoding. arXiv:2604.14862, arXiv preprint; accessed 2026-09-18, 2604.14862
- Li Y (2026) When JSON is not enough: Semantic reliability of schema-constrained LLM ordering agents. arXiv:2607.18261, arXiv preprint; accessed 2026-09-28, 2607.18261
- Ligatti J, Bauer L, Walker D (2005) Edit automata: Enforcement mechanisms for runtime security policies. *International Journal of Information Security* 4(1–2):2–16, DOI 10.1007/s10207-004-0046-8
- Liu J (2023) Instructor: Structured outputs for LLMs. <https://github.com/instructor-ai/instructor>, software; validation context and re-asking on failure
- Lundberg S, Ribeiro MT (2023) Guidance: A guidance language for controlling large language models. <https://github.com/guidance-ai/guidance>
- Luo Q, Nahian SMA, Gutttag J, Abulnaga SM, Matton K (2026) A removal based approach to improve LLM faithfulness at test-time. arXiv:2609.04343, 2609.04343
- Maynez J, Narayan S, Bohnet B, McDonald R (2020) On faithfulness and factuality in abstractive summarization. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp 1906–1919, DOI 10.18653/v1/2020.acl-main.173
- Meyer B (1992) Applying “design by contract”. *Computer* 25(10):40–51
- Muchlinski D, Siroky D, He J, Kocher M (2016) Comparing random forest with logistic regression for predicting class-imbalanced civil war onset data. *Political Analysis* 24(1):87–103
- Ng AW, Han Y, Zhang J, Wang W (2026) Agent safety should be a runtime contract. arXiv:2608.11274, 2608.11274
- OpenAI (2024) Structured outputs. <https://platform.openai.com/docs/guides/structured-outputs>
- Penrose KW, Nelson AG, Fisher AG (1985) Generalized body composition prediction equation for men using simple measurement techniques. *Medicine and Science in Sports and Exercise* 17(2):189
- Poesia G, Polozov O, Le V, Tiwari A, Soares G, Meek C, Gulwani S (2022) Synchromesh: Reliable code generation from pre-trained language models. In: International Conference on Learning Representations (ICLR)
- Ralph P, et al. (2020) ACM SIGSOFT empirical standards for software engineering research. arXiv:2010.03525

- Rebedea T, Dinu R, Sreedhar MN, Parisien C, Cohen J (2023) NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations, pp 431–445, DOI 10.18653/v1/2023.emnlp-demo.40
- Reiter R (1978) On closed world data bases. In: Gallaire H, Minker J (eds) Logic and Data Bases, Plenum Press, pp 55–76
- Sabuniş H, Ünlü Y, Özdemir MK (2026) mlcompass: Artifact and replication package. Zenodo, version v0.10.0, doi: 10.5281/zenodo.23025968, DOI 10.5281/zenodo.23025968, source code: <https://github.com/hakansabunis/mlcompass>
- Schick T, Dwivedi-Yu J, Dessi R, Raileanu R, Lomeli M, Zettlemoyer L, Cancedda N, Scialom T (2023) Toolformer: Language models can teach themselves to use tools. In: Advances in Neural Information Processing Systems (NeurIPS), 2302.04761
- Schneider FB (2000) Enforceable security policies. ACM Transactions on Information and System Security 3(1):30–50, DOI 10.1145/353323.353382
- Scholak T, Schucher N, Bahdanau D (2021) PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 9895–9901
- Sclar M, Choi Y, Tsvetkov Y, Suhr A (2024) Quantifying language models’ sensitivity to spurious features in prompt design. In: International Conference on Learning Representations (ICLR), 2310.11324
- Signé Q, Boughanem M, Moreno J, Belkacem T (2026) Guaranteeing faithful evidence extraction in speculative retrieval-augmented generation. arXiv:2609.10046, 2609.10046
- Truong O, Zhang T, Marchareddy A, Lee R, Busold J, Socas M, AlOmar EA (2025) LeakageDetector 2.0: Analyzing data leakage in jupyter-driven machine learning pipelines. arXiv:2509.15971, 2509.15971
- Wang Y, Zhang J, Wu Z, Cai T, Liu Z, Sun Z, Dong M, Zheng M, Duan Y, Yin X, Zhu Y (2026) From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. arXiv:2606.04990, 2606.04990
- Willard BT, Louf R (2023) Efficient guided generation for large language models. arXiv:2307.09702, 2307.09702
- Winston C, Winston C, Just R (2026) Solver-aided verification of policy compliance in tool-augmented LLM agents. arXiv:2603.20449, arXiv preprint; accessed 2026-09-27, 2603.20449
- Wiseman S, Shieber S, Rush A (2017) Challenges in data-to-document generation. In: Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 2253–2263
- Xu B (2025) Hallucination is inevitable for LLMs with the open world assumption. arXiv:2510.05116, 2510.05116
- Yang C, Brower-Sinning RA, Lewis GA, Kästner C (2022) Data leakage in notebooks: Static detection and better processes. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)
- Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y (2023) ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (ICLR), 2210.03629
- YData (2026) ydata-profiling: Automated exploratory data analysis for pandas dataframes. <https://github.com/ydataai/ydata-profiling>, software repository. Accessed 14 September 2026
- Zaharia M, Chen A, Davidson A, Ghodsi A, Hong SA, Konwinski A, Murching S, Nykodym T, Ogilvie P, Parkhe M, Xie F, Zumar C (2018) Accelerating the machine learning lifecycle with MLflow. IEEE Data Engineering Bulletin 41(4):39–45, URL <https://dblp.org/db/journals/debu/debu41.html>
- Zhang S, Zhao J, Dong H, Xu R, Li Z, Zhang Y, Li S, Wen Y, Xia C, Wang Z, Feng X, Cui H (2025a) When grammar guides the attack: Uncovering control-plane vulnerabilities in LLMs with structured output. arXiv:2503.24191, arXiv preprint; accessed 2026-09-28, 2503.24191
- Zhang Y, Emma SY, En ALJ, Dong JS (2025b) RvLLM: LLM runtime verification with domain knowledge. arXiv:2505.18585, arXiv preprint; accessed 2026-09-27, 2505.18585